COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 031	**SECTION:**	CPE21S4
DATE PERFORMED:	8/5/2026	**DATE SUBMITTED:**	8/8/2026
NAME:	Jerry Matthew L. Larracas	**INSTRUCTOR:**	Engr. Jimlord Quejado

ASSIGNMENT NO. 1.1 **Data Cleaning with Python**

1. PROCEDURE OUTPUT

ILOA: Create queue using C++ STL

Code

HOA 5 Queues > ILOA > ILOAmain.cpp > main()

```cpp
#include <iostream>
#include <queue>

void display(std::queue <int> q){
    std::queue <int> c = q;
    while (!c.empty()){
        std::cout<< " " << c.front();
        c.pop();
    }
    std::cout << "\n";
}

int main(){
    std::queue <int> a;
    a.push(10);
    a.push(20);
    a.push(30);

    std::cout << "The queue a is: ";
    display(a);

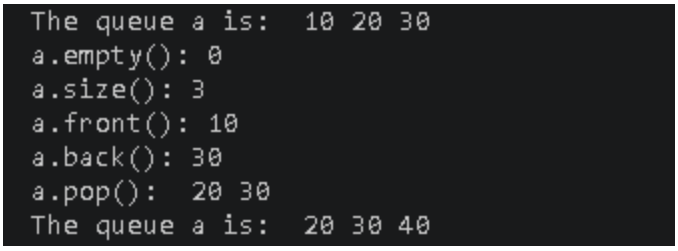

    std::cout<< "a.empty(): " << a.empty() << "\n";
    std::cout<< "a.size(): " << a.size() << "\n";
    std::cout<< "a.front(): " << a.front() << "\n";
    std::cout<< "a.back(): " << a.back() << "\n";

    std::cout<< "a.pop(): ";
    a.pop();
    display(a);

    a.push(40);
    std::cout << "The queue a is: ";
    display(a);

    return 0;

}
```

Output	
Observation	In this part we implement a queue using C++ STL which means we have its operations such as empty for checking if the queue is empty, the size checks how many are in queueing, front checks if what value is at the front, back checks if what value is at the back and the pop will simply just delete the queueing at the front.

ILOB Part 1

Code

Main file

```cpp
#include <iostream>
#include "queueLL.h"
#include "queueArr.h"

int main(){

    qNode<int>* front = nullptr;
    qNode<int>* back = nullptr;

    enqueue(1, &front, &back);
    std::cout << "Front: " << front->data <<" back: " << back->data << "\n";
    enqueue(2, &front, &back);
    enqueue(3, &front, &back);
    std::cout << "Front: " << front->data <<" back: " << back->data << "\n";

    //display()
    display(front);

    dequeue(&front, &back);
    std::cout << "Front: " << front->data <<" back: " << back->data << "\n";

    dequeue(&front, &back);
    std::cout << "Front: " << front->data <<" back: " << back->data << "\n";
    dequeue(&front, &back);
    dequeue(&front, &back);
```

queueLL.h file

HOA 5 Queues > ILOB > queueLL.h > ...

```h
#ifndef QUEUELL_H
#define QUEUELL_H
#include <iostream>

template <typename T>
class qNode{
    public:
    T data;
    qNode *next;
};

template <typename T>
qNode<T> *new_node(T newData){
    qNode<T> *newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

template<typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
    //create a new node
    qNode<T>* newNode = new_node(newData);

    //insterting an item into a empty queue
    if((*frontPtr) == nullptr && (*backPtr)== nullptr){
        //assign the frontPtr to the newNode
        (*frontPtr) = newNode;
        //assign the backPtr to the newNode
        (*backPtr) = newNode;
    }

   //point the back nex to the newNode
   (*backPtr)->next = newNode;
   // the newNode will be the new backPtr
   (*backPtr) = newNode;
   return;


}

template<typename T>
void dequeue(qNode<T> **frontPtr, qNode<T> **backPtr){
    //how to determine if the queue is empty?
    if((*frontPtr)== nullptr && (*backPtr) == nullptr){
        return;
    }

    //creating a temporary pointer
    qNode<T>* temPtr = nullptr;
    //store the frontPtr to the tempPrt
    temPtr = (*frontPtr);

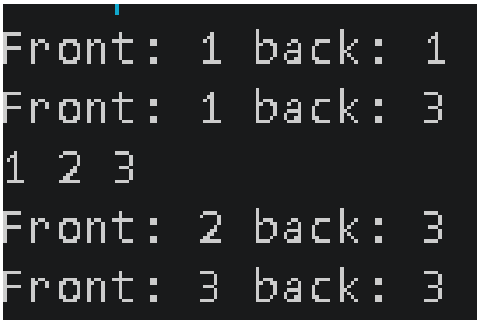
    //check if queue has only 1 node left?
    if((*frontPtr) == (*backPtr)){
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete temPtr;
        return;
    }

    //update the frontPtr
    (*frontPtr) = (*frontPtr)->next;

    //point the temPtr to next to null
    temPtr->next = nullptr;

    delete temPtr;
}

//display all items in the queue
template <typename T>
void display(qNode<T>* frontPtr){
    //catch if the queue is empty
    if(frontPtr == nullptr){
        std::cout << "The queue is empty.\n";
        return;
    }
    //temp pointer to store the front Ptr
    qNode<T>* current = frontPtr;
    //create a loop that will print the temPtr->data
    while(current != nullptr){
        std::cout << current->data << " ";
        current = current->next;
    }
    std::cout << std::endl;
}
#endif
```

Output	
Observation	The output shows the function of the enqueueing of the values from front to back. The first output line enqueues only 1 value that is why it shows that front and back has only 1 value. Second output line we enqueue 2 and 3 value and then shows the front is 1 and the back is 3 because 3 is at the last queue or back. The third output line simply just displays all of the values that are in queue. In fourth output line this time we will dequeue or remove the current value of the front and it will replace the next value which is 2. And last output line, dequeue again, it will not delete the 2 at the front replacing with 3 that was on queue recently and now it will show that the front is now 3 and the back is also 3 because 3 is the only value that is currently existing.

ILOB Part 2

Code	Main file
	``
	//Object creation
	
	queueArr<int> q(6);
	
	q.Enqueue(1);
	std::cout << "Front: " << q.Front() <<" back: " << q.Back() << "Size: "<< q.Size() << "\n";
	q.Enqueue(2);
	std::cout << "Front: " << q.Front() <<" back: " << q.Back() << "Size: "<< q.Size() << "\n";
	q.Enqueue(3);
	std::cout << "Front: " << q.Front() <<" back: " << q.Back() << "Size: "<< q.Size() << "\n";
	q.Enqueue(4);
	std::cout << "Front: " << q.Front() <<" back: " << q.Back() << "Size: "<< q.Size() << "\n";
	q.Enqueue(5);
	std::cout << "Front: " << q.Front() <<" back: " << q.Back() << "Size: "<< q.Size() << "\n";
	q.Enqueue(6);
	std::cout << "Front: " << q.Front() <<" back: " << q.Back() << "Size: "<< q.Size() << "\n";
	
	return 0;
	``

queueArr.h file

HOA 5 Queues > ILOB > C queueArr.h > ~queueArr()

```h
#ifndef QUEUEARR_H
#define QUEUEARR_H
#include <iostream>

template <typename T>
class queueArr{
    private:
    //pointer to dynamically alalocated array
        T* q_array;
        size_t q_capacity;
        size_t q_size;
        int q_front;
        int q_back;

    public:
    //constructor
    queueArr(size_t capacity = 10);

    bool Full();
    bool Empty();
    size_t Size();
    void Clear();

    T Front();
    T Back();

    void Enqueue(T value);
    void Dequeue();

    //Copy constructor
    queueArr(const queueArr& other);

    //Copy assignment operator
    queueArr& operator=(const queueArr& other);

    //destructor
    ~queueArr();


};

//constructor
template<typename T>
queueArr<T>::queueArr(size_t capacity){
    q_capacity = capacity;
    q_size = 0;
    q_front = 0;
    q_back = -1;
    q_array = new T(q_capacity);

}

template<typename T>
bool queueArr<T>::Full(){
    return q_size == q_capacity;
}

template<typename T>
bool queueArr<T>::Empty(){
    return q_size == 0;
}

template<typename T>
size_t queueArr<T>::Size(){
    return q_size;
}

template<typename T>
void queueArr<T>::Clear(){
    q_size = 0;
    q_front = 0;
    q_back = -1;
}

template<typename T>
T queueArr<T>::Front(){
    // what will happen if the queue is empty
    if(Empty()){
        std::cout << "The Queue is Empty. \n";
        return T();

    }
    return q_array[q_front];

}

```

```
86
87  template <typename T>
88  T queueArr<T>::Back(){
89      if(Empty()){
90          std::cout << "The Queue is Empty. \n";
91          return T();
92      }
93      return q_array[q_back];
94  }
95
96  template<typename T>
97  void queueArr<T>::Enqueue(T value){
98      //check if the queue is full
99      if(Full()){
100         std::cout << "The Queue is Full.\n";
101         return;
102     }
103     //compute for the q_back for circular (use the module operator)
104     q_back = (q_back + 1) % q_capacity;
105
106     //assign the q_array with the updated q_back to the value
107     q_array[q_back+1] = value;
108     //increment the q_size
109     q_size++;
110     return;
111 }
112
113 template<typename T>
114 void queueArr<T>::Dequeue(){
115     //check if it is empty
116     if(Empty()){
117         std::cout << "The Queue is Empty.\n";
118         return;
119     }
120     //move q_front circularity
121     q_front = (q_front + 1) % q_capacity;
122     //decrement the q_size
123     q_size--;
124
125     if(Empty()){
126         Clear();
127         return;
128     }
129 }
130
131 template<typename T>
132 queueArr<T>::~queueArr(){
133     delete[] q_array;
134 }
135 #endif //QUEUEARR_LL
```

Output

```
Front: 6 back: 6 Size: 1
Front: 6 back: 1 Size: 2
Front: 6 back: 2 Size: 3
Front: 6 back: 3 Size: 4
Front: 6 back: 4 Size: 5
Front: 6 back: 5 Size: 6
```

Observation	This demonstrates how queue works using circular array. When the values 1 to 4 are added to the queue the size increases from 1 to 6, showing that each value are being stored correctly. The front and back values also change as values are added, in which helps show how the queue keeps track of its first and last values. This queuing array includes functions such as enqueue, dequeue, front, back, size, full, and empty to manage the queue.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **n\a**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **n\a**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **n\a**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.